

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 1

Duration :60 Mins

On Class

Total Marks:50

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I A K Selva Prabhu(192421224)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A.K. Selva



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Language Processors : Compiler vs Interpreter

CSA1403 – COMPILER DESIGN

Presented By:

A K Selva Prabhu

192421224

Supervised by:

Dr . Michael George

INTRODUCTION

What is a Language Processor ?

A **language processor** is system software that translates a program written in a **high-level language** (such as C, Java, or Python) into **machine language** (binary code) so that the computer can understand and execute it.

Why is it Needed?

- Computers understand only machine language (0s and 1s).
- Programmers write code in human-readable languages.
- A language processor acts as a bridge between the programmer and the compute

TYPES OF LANGUAGE PROCESSORS

There are three main types of Language Processors:

1. Compiler:

- Translates the **entire source program** into machine code at once.
- Generates an **executable file**.
- Reports errors after the compilation process is complete.

2. Interpreter:

- Translates and executes the program **one statement at a time**.
- Does **not** create a separate executable file.
- Stops execution immediately when an error is encountered.

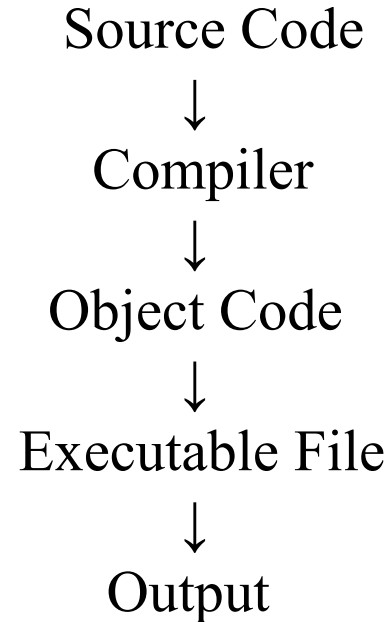
3. Assembler:

- Converts **assembly language** into machine language.
- Produces object code that can be executed by the computer.
- Used in low-level system programming and embedded systems

COMPILER

A **Compiler** is a language processor that translates the **entire source code** into machine code **before execution**. If the program is error-free, it generates an executable file.

WORKING PROCESS:



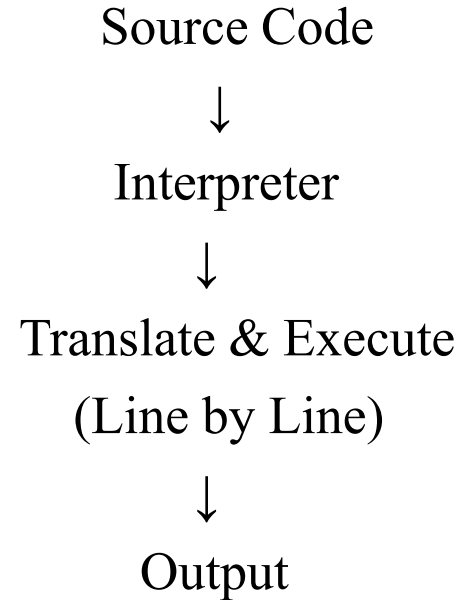
Examples:

C , C++ , Go

INTERPRETER

An **Interpreter** is a language processor that **translates and executes the program one statement at a time**. It does not generate a separate executable file.

WORKING PROCESS:



Examples:

Python , JavaScript

COMPARISON

FEATURES	COMPILER	INTERPRETER
Execution	Executes after successful compilation	Executes immediately after translating each line
Executable File	Executable File	Does not create an executable file
Error Handling	Displays all errors after compilation	Displays errors one by one during execution
Execution Speed	Faster	Slower
Memory Usage	More memory required	Less memory required

REAL WORLD EXAMPLE AND APPLICATIONS

Applications of a Compiler:

- Operating Systems
- Desktop Applications
- Game Development
- Embedded Systems
- System Software

Applications of an Interpreter:

- Web Development
- Data Science and Machine Learning
- Automation Scripts
- Rapid Prototyping
- Interactive Programming

REAL-LIFE ANALOGY:

Compiler

Like translating an entire **book** into another language before anyone reads it. Translation is done once, and the translated book can be read many times.

Interpreter

Like translating a **speech** sentence by sentence while the speaker is talking. Translation happens continuously as the speech is delivered.

CONCLUSION

- Language processors are essential for executing programs written in high-level languages.
- **Compilers** are best suited for high-performance applications.
- **Interpreters** are ideal for scripting, testing, and rapid development.
- Understanding both helps programmers choose the right tool for different programming tasks.
- The choice between a compiler and an interpreter depends on the application's performance and development needs.
- Knowing their strengths and limitations helps developers choose the most suitable approach for software development.

THANK YOU

Presentation Photo :

